- **滚动事件**
  - 语法：
    - element.addEventListener('scroll', function () {});
  - 触发方式：滚动条滚动的时候触发
    - 事件要添加给事件源、那如何为页面滚动添加事件
    - 可以为document添加上事件、也可以给window添加事件
    - 如果某个元素内容移除要添加滚动事件，就给元素添加
  - 总结：
    - 监听页面的滚动，window 和 document 均可监听该事件
    - 在滚动监听的过程中能实时获取页面滚动的距离

- **加载事件**
  - 语法：
    - element.addEventListener('load',function () {});
  - 说明：
    - 网页当中凡是写路径的都会再network被查看
    - 凡是有路径的都需要有外部资源
  - 注意：
    - 通过window来监听页面加载的情况
    - window.addEventListener('load', function () {});
    - 页面所以的外部资源加载完成才会触发、和顺序无关

- **节点大小**
  - 语法：
    - element.clientWidth、element.offsetWidth、element.scrollWidth
  - 注意：
    - 以上方法获取的元素大小是数字类型，可以直接用于计算
    - content、padding、border ，在使用 JavaScript 动态获取 DOM 元素大小时同样需要从这 3 个方面考虑
  - 结论：
    - clientWidth/Height：content + padding
    - offsetWidth/Height：content + padding + border
    - scrollWidth/Height：content + padding + 溢出
  - 拓展：
    - Element.getBoundingClientRect() 获取相对于浏览器窗口左上角的位置及元素的大小

- **节点位置**
  - 语法：
    - 元素.offsetTop、元素.offsetLeft
  - 结论：

        - offsetLeft/Top 相对最近的已定位的祖先元素的位置，当所有祖先都未定位则参照 body 元素，fixed 定位的元素始终参照浏览器窗口。
        - offsetParent 获取元素位置时所参照的祖先元素
    - 拓展：
        - IntersectionObserver判断元素是否在视窗内

- **节点卷起位置**
    - 语法：
        - 元素.scrollTop、元素.scrollLeft
    - 结论：
        - scrollLeft/Top 动态获取元素滚动的位置
        - scrollLeft/Top 允许被赋值动态改变元素的滚动位置
        - 获取整个页面的滚动位置通过 documentElement 元素
        - 获取的是数字类型的之值，这个属性可以设置
    - 扩展：
        - 还有一对属性 clientLeft 和 clientTop 实际指的是边框的宽度

- **延迟函数**
    - 一次性定时器：
        - 延迟性定时器，可以让一段代码延迟执行
    - 语法：
        - window.setTimeout(function () {}, 时间)
    - 说明：
        - 所有非延迟代码执行完了才会开始执行延迟代码（即使延迟事件为零），所以延迟函数的实际执行时间会有误差
        - 延时函数和间歇函数统称为定时器或定时任务
        - 结合递归函数可以使用 setTimeout 实现 setInterval 一样的功能
        - setTimeout 结合递归函数创建定时任务时，是立即执行定的，因此实践中经常使用 setTimeout 配合递归来替代 setInterval

- **清除延迟函数**
    - 语法：window.clearTimeout（定时器名）
    - 注意：定时器代码和非定时器代码

- **动画**
    - JS动画：
        - 创建定时任务不间断的更改元素的样式，实现动画效果
        - 借助定时器创建动画效时常使用 setTimeout
    - requestAnimationFrame：

    - requestAnimationFrame 是一个类似 setTimeout 的函数，在频繁操作 DOM 时的性能更好，但是浏览器的兼容性较setTimeout 稍差。
  - 总结：
    - 频繁操作 DOM 时，requestAnimationFrame 比 setTimeout 和 setInterval 性能更好
    - 基于 JavaScript DOM 操作实现的动画多采用 requestAnimationFrame
    - cancelAnimationFrame 停止执行 requestAnimationFrame